Détection automatique des plantes dans les cultures

Buc Jade

N°de candidat : 39633

Comment concevoir un modèle de reconnaissance d’images permettant à un robot agricole d’identifier les mauvaises herbes dans les cultures ?

Plan

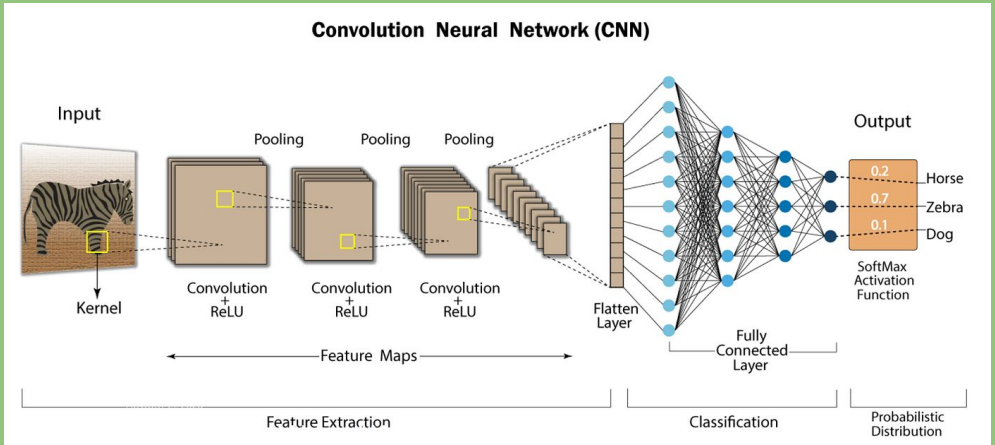
1- Le traitement d’images et les CNN (réseau de neurones convolutifs)

2- Résultats et observations

3- Analyse critique du projet

1- Le traitement d’images et les CNN

```python
00000001 import torch
00000002 import torch.nn as nn
00000003 import torch.nn.functional as F
00000004
00000005 print(torch.__version__)
00000006 print(torch.cuda.is_available())
00000007
00000008
00000009 class CNN(nn.Module):
00000010     def __init__(self):
00000011         super().__init__()
00000012         self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
00000013         self.bn1 = nn.BatchNorm2d(32)
00000014
00000015         self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
00000016         self.bn2 = nn.BatchNorm2d(64)
00000017
00000018         self.pool = nn.MaxPool2d(2, 2)
00000019         self.dropout = nn.Dropout(0.5)
00000020
00000021         self.fc1 = nn.Linear(64 * 32 * 32, 128)
00000022         self.fc2 = nn.Linear(128,6)
00000023
00000024     def forward(self, x):
00000025         x = self.pool(F.relu(self.bn1(self.conv1(x))))
00000026         x = self.pool(F.relu(self.bn2(self.conv2(x))))
00000027
00000028         x = x.view(-1, 64 * 32 * 32)
00000029         x = self.dropout(F.relu(self.fc1(x)))
00000030         x = self.fc2(x)
00000031
00000032         return x
```



source : nafizshahriar.medium.com

1-

Prétraitement des images

- **Standardisation des formats** : toutes les images du dataset ont été converties en format RGB, puis enregistrées en `.png` ou `.jpg` afin d'assurer une compatibilité uniforme avec le modèle d'apprentissage.

- **Rééchantillonnage** : Les images ont été redimensionnées à une taille fixe de 128 × 128 pixels.

- **Correction des écarts visuels** : Des ajustements ont été appliqués pour corriger les différences de couleur, de saturation et d'exposition entre les images, afin de standardiser leur apparence visuelle et de faciliter l'apprentissage du modèle.

1-

Prétraitement des images

Répartition des données :

Le dataset a été divisé en deux sous-ensembles :

- **80 % des images** (640 images par classe) ont été utilisées pour l'**entraînement** du modèle.
- **20 % restantes** (160 images par classe) ont servi au **test**, c'est-à-dire à l'évaluation des performances sur des données jamais vues par le modèle.

Normalisation des images :

Avant d'être transmises au modèle, les valeurs des pixels de chaque image ont été **normalisées** à l'aide de la formule :

$$\text{valeur normalisée} = \frac{\text{valeur - moyenne}}{\text{écart-type}}$$

1-

Entraînement

- **Fonction de coût** : `CrossEntropyLoss`, mesure l'écart entre les prédictions du modèle et les vraies classes associées aux images. Cette fonction est adaptée à la classification multi-classes, car elle pénalise fortement les mauvaises prédictions.

- **Optimisation** : Le modèle est entraîné grâce à l'algorithme d'optimisation **Adam** (Adaptive Moment Estimation), avec un taux d'apprentissage fixé à 0.001.

- **Entraînement par lots** : L'apprentissage se fait par **mini-lots de 32 images** (batch size = 32). Ce découpage permet une meilleure efficacité mémoire et une stabilisation de la descente du gradient, tout en maintenant une bonne vitesse d'entraînement.

- **Régularisation** : Utilisation de la technique **Dropout** pour limiter le **surapprentissage**. Cette méthode consiste à désactiver aléatoirement certains neurones pendant l'apprentissage, ce qui oblige le modèle à ne pas dépendre d'un chemin unique dans le réseau et améliore sa capacité de généralisation.

1-

Test

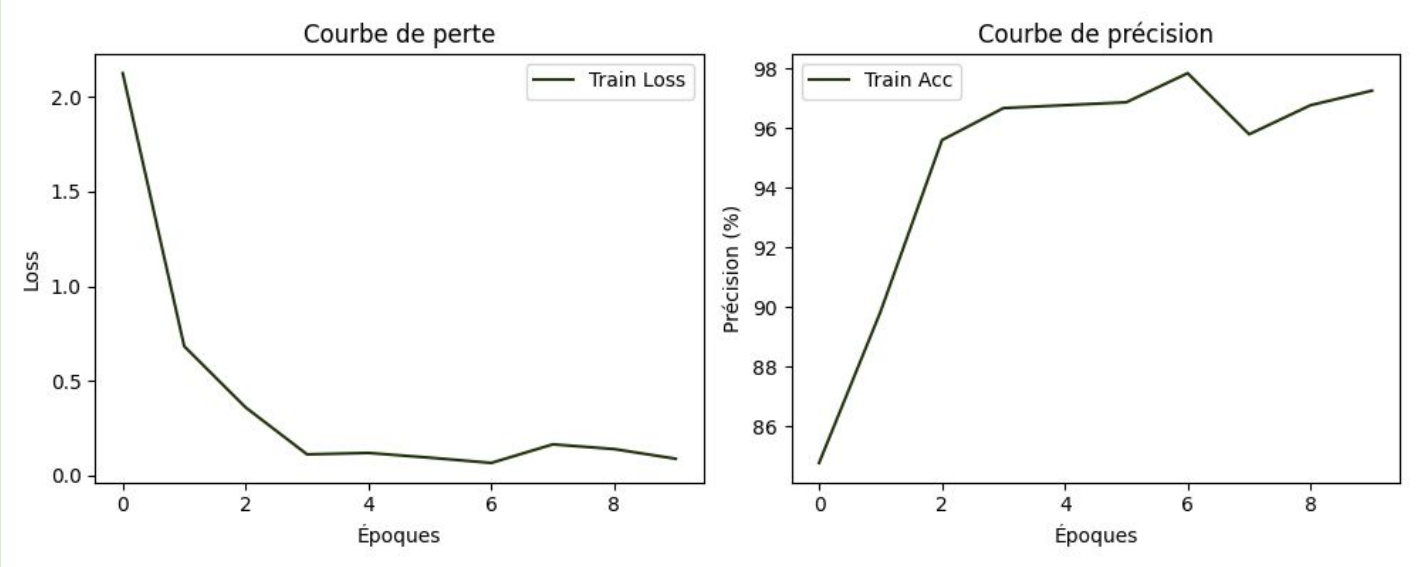
- **Passage en mode évaluation** : Avant d'évaluer les performances du modèle, on le place en mode évaluation. Cette étape est essentielle car elle désactive le dropout et le calcul automatique des gradients.

- **Évaluation finale du modèle** : Le modèle est testé sur un ensemble d'images qu'il n'a jamais vues auparavant. Le modèle effectue une prédiction pour chaque image et compare le résultat avec la vraie classe.

Objectif : Cette phase permet de mesurer les performances réelles du modèle, c'est-à-dire sa capacité à généraliser ce qu'il a appris à de nouvelles données.

2- Résultats et observations

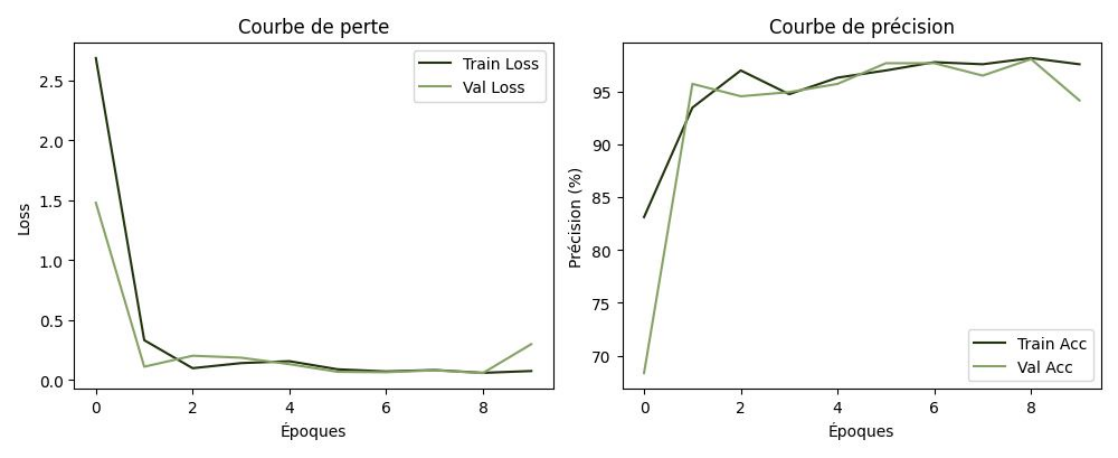
Entraînement



2-

Surapprentissage

- **validation loss** : erreur moyenne du modèle sur les données qu'il n'a pas apprises.
- **validation accuracy** : pourcentage d'images correctement classées sur le jeu de validation.



2-

Test

nombre de catégories	Précision (en %)	catégorie ajoutée
2	97,50	pommier/myrtillier
3	97,50	cerisier
4	93,12	soja
5	91,62	tomate
6	95,42	mais
7	93,84	poivron
8	89,84	vigne
9	87,43	fraisier
10	87,59	oranger

Maïs

Tomate

2-

Transformations

	Dataset propre (1600 images)	Dataset bruité (50 images)
Transformations	précision (en %)	
Aucune	98.57	70
Flouté	93.93	60
Couleurs (fort)	96.43	60
Couleurs (léger)	96.43	70
Dropout	95.83	70
Rotation et miroir	91.07	90
Tout combiné	91.07	80

3- Analyse critique du projet

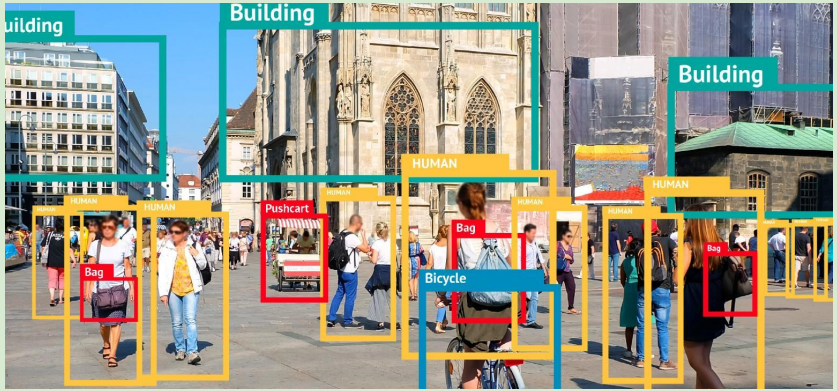
Limites rencontrées dans mon projet

- **Choix et taille du dataset** : Les datasets utilisés sont soit relativement réduits en volume, soit peu diversifiés. Cela limite la capacité du modèle à apprendre des caractéristiques variées, et donc à bien généraliser à des cas réels très différents .
- **Taille fixe des images** : Toutes les images ont été échantillonnées à une taille unique. Cette étape entraîne aussi une perte de détails.
- **Gestion des arrière-plans complexes** : Le modèle a été entraîné sur des images où le fond était uniforme ou contrasté. Cela entraîne une confusion dans la prédiction, notamment lorsque plusieurs plantes ou objets parasites apparaissent dans l'image.
- **Overfitting possible** : Le modèle atteint parfois une très bonne précision sur les données d'entraînement, mais subit une chute significative sur les données de test.

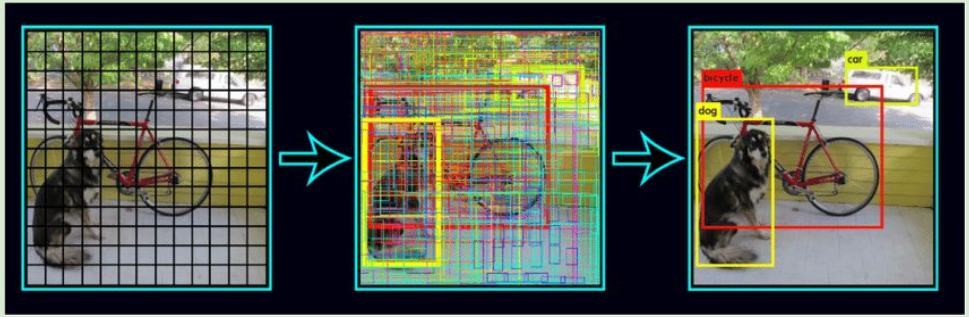
Piste d’amélioration

Utilisation d’un modèle **YOLO** (You Only Look Once) :

- Meilleure détection d’objets en contexte réel.
- Capacité à gérer plusieurs classes et arrière-plans variés.
- Une meilleure précision et une rapidité accrue, même sur des images haute résolution.



Source : *Blent,* juin 2022



Source : October 2020- EURASIP Journal

Conclusion

Entraînement 1/4

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
from cnn import CNN

import random
import numpy as np
import matplotlib.pyplot as plt

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

# Transformations
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
])
```

Entraînement 2/4

```
00000030 # Chargement du dataset et séparation train/val
00000031 full_dataset = datasets.ImageFolder('datasets/train/images' , transform=transform)
00000032 train_size = int(0.8 * len(full_dataset))
00000033 val_size = len(full_dataset) - train_size
00000034 train_dataset , val_dataset = random_split (full_dataset , [train_size , val_size])
00000035
00000036 train_loader = DataLoader (train_dataset , batch_size=32, shuffle=True)
00000037 val_loader = DataLoader (val_dataset , batch_size=32, shuffle=False)
00000038
00000039 # Modèle, perte, optimiseur
00000040 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
00000041 model = CNN().to(device)
00000042 criterion = nn.CrossEntropyLoss()
00000043 optimizer = optim.Adam(model.parameters(), lr=0.001)
00000044
00000045 # Suivi des courbes
00000046 train_losses = []
00000047 train_accuracies = []
00000048 val_losses = []
00000049 val_accuracies = []
00000050
00000051 # Entraînement
00000052 for epoch in range(10):
00000053     model.train()
00000054     total_loss = 0.0
00000055     nb_bonnes = 0
00000056     total = 0
00000057
```

Entraînement 3/4

```
00000058    for images, labels in train_loader:
00000059        images, labels = images.to(device), labels.to(device)
00000060        optimizer.zero_grad()
00000061        outputs = model(images)
00000062        loss = criterion(outputs, labels)
00000063        total_loss += loss.item()
00000064        loss.backward()
00000065        optimizer.step()
00000066
00000067        _, predicted = torch.max(outputs, 1)
00000068        nb_bonnes += (predicted == labels).sum().item()
00000069        total += labels.size(0)
00000070
00000071    train_accuracy = 100 * nb_bonnes / total
00000072    train_loss = total_loss / len(train_loader)
00000073    train_losses.append(train_loss)
00000074    train_accuracies.append(train_accuracy)
00000075
00000076    # Validation
00000077    model.eval()
00000078    val_loss = 0.0
00000079    val_correct = 0
00000080    val_total = 0
00000081
```

Entraînement 4/4

```
00000082     with torch.no_grad():
00000083         for images, labels in val_loader:
00000084             images, labels = images.to(device), labels.to(device)
00000085             outputs = model(images)
00000086             loss = criterion(outputs, labels)
00000087             val_loss += loss.item()
00000088             _, predicted = torch.max(outputs, 1)
00000089             val_correct += (predicted == labels).sum().item()
00000090             val_total += labels.size(0)
00000091
00000092     val_accuracy = 100 * val_correct / val_total
00000093     val_losses.append(val_loss / len(val_loader))
00000094     val_accuracies.append(val_accuracy)
00000095
00000096     print(f"Epoch {epoch+1}/10")
00000097     print(f" → Train Loss: {train_loss:.4f}, Accuracy: {train_accuracy:.2f}%")
00000098     print(f" → Val   Loss: {val_losses[-1]:.4f}, Accuracy: {val_accuracy:.2f}%\n")
00000099
00000100 plt.figure(figsize=(10, 4))
00000101
00000102 plt.subplot(1, 2, 1)
00000103 plt.plot(train_losses, label='Train Loss',color='#22340d')
00000104 plt.plot(val_losses, label='Val Loss',color="#87a266")
00000105 plt.xlabel('Époques')
00000106 plt.ylabel('Loss')
00000107 plt.title('Courbe de perte')
00000108 plt.legend()
00000109
```

```
00000110 plt.subplot(1, 2, 2)
00000111 plt.plot(train_accuracies, label='Train
Acc',color="#22340d")
00000112 plt.plot(val_accuracies, label='Val Acc',color="#87a266")
00000113 plt.xlabel('Époques')
00000114 plt.ylabel('Précision (%)')
00000115 plt.title('Courbe de précision')
00000116 plt.legend()
00000117
00000118 plt.tight_layout()
00000119 plt.show()
```

Test 1/2

```
00000001 import torch.nn as nn
00000002 import torch.nn.functional as F
00000003 import torch.optim as optim
00000004 from torchvision import datasets, transforms
00000005 from torch.utils.data import DataLoader
00000006 import torch
00000007
00000008 from cnn import CNN
00000009
00000010 # Transformation des images
00000011 transform = transforms.Compose([
00000012     transforms.Resize((128, 128)),
00000013     transforms.ToTensor(),
00000014     transforms.Normalize([0.5,0.5,0.5],[0.5,0.5,0.5])
00000015 ])
00000016
00000017 # Chargement des données de test
00000018 test_dataset = datasets.ImageFolder('datasets/test/images',
transform=transform)
00000019 test_loader = DataLoader(test_dataset, batch_size=5, shuffle=False)
# pas de mélange pour évaluer
00000020
00000021 #Choix du support
00000022 device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
00000023 modele = CNN().to(device)  # on envoie le modèle sur le GPU ou CPU
00000024 modele.load_state_dict(torch.load("cnn.pth",map_location=device))
00000025 # Définition de la fonction de coût et de l'optimiseur
00000026 critere = nn.CrossEntropyLoss()
00000027 optimiseur = optim.Adam(modele.parameters(), lr=0.001)
```

Test 2/2

```python

# évaluation
correct = 0
total = 0
modele.eval()

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = modele(images)
        _, predi = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predi == labels).sum().item()
print(test_dataset.class_to_idx)


print(f"Accuracy: {100 * correct / total:.2f}%")


torch.save(modele.state_dict(), "cnn.pth")
```

Fenêtre pour tester une seule image 1/2

```
00000001 import tkinter as tk
00000002 from tkinter import filedialog
00000003 from PIL import Image, ImageTk
00000004 import torch
00000005 from torchvision import transforms
00000006 from cnn import CNN
00000007
00000008 # choix du support
00000009 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
00000010
00000011 # Chargement du modèle
00000012 model = CNN().to(device)
00000013 model.load_state_dict(torch.load("cnn.pth", map_location=device))
00000014 model.eval()
00000015
00000016 classes = {0: 'pommier', 1: 'myrtille'}
00000017
00000018 transform = transforms.Compose([
00000019     transforms.Resize((128, 128)),
00000020     transforms.ToTensor(),
00000021     transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
00000022 ])
00000023
00000024 # Fonction de chargement et de prédiction
00000025 def charger_image():
00000026     chemin = filedialog.askopenfilename()
00000027     if not chemin:
00000028         return
00000029
```

Fenêtre pour tester une seule image 2/2

```
00000030     img = Image.open(chemin).convert('RGB')
00000031     input_tensor = transform(img).unsqueeze(0).to(device)
00000032
00000033     with torch.no_grad():
00000034         output = model(input_tensor)
00000035         _, prediction = torch.max(output.data, 1)
00000036         resultat.set(f"Résultat : {classes[prediction.item()]}")
00000037
00000038 # Interface graphique
00000039 fenetre = tk.Tk()
00000040 fenetre.title("Pommier ou Cerisier ?")
00000041
00000042 bouton = tk.Button(fenetre, text="Choisir une image", command=charger_image)
00000043 bouton.pack(pady=10)
00000044
00000045 resultat = tk.StringVar()
00000046 label_resultat = tk.Label(fenetre, textvariable=resultat, font=("Arial", 20))
00000047 label_resultat.pack(pady=10)
00000048
00000049 fenetre.mainloop()
```